Project 5 - Arrays, Addressing, and Stack-Passed Parameters

Start Assignment

- Due Nov 24 by 11:59pm
- Points 50
- Submitting a file upload
- File Types asm
- Available after Nov 10 at 12am

Introduction

This program is significantly more difficult than previous programs, partially due to the more advanced concepts. The Rubric (see below) has a number of point deductions for not meeting requirements. It is not uncommon for a student to generate a program that meets the **Program Description** but violates several **Program Requirements**, causing a *massive* loss in points. Please carefully review the Rubric Requirements to avoid this circumstance.

The purpose of this assignment is to reinforce concepts related to usage of the runtime stack, proper modularization practices, use of the STDCALL calling convention, use of arrays, and memory addressing modes (CLO 3, 4, 5).

1. Using Indirect Operands, Register Indirect, and/or Base+Offset addressing
2. Passing parameters on the runtime stack
3. Generating "random" numbers
4. Working with arrays

What you must do

Program Description

Write and test a MASM program to generate a series of temperature readings (`TEMPS_PER_DAY` readings per day for `DAYS_MEASURED` days) in the range of [`MIN_TEMP` ... `MAX_TEMP`] into one array. Once this array is generated, calculate the highest and lowest temperature reading for each day, then calculate the average of each of these daily high and low temperatures. Display all this information with descriptive titles. More info on this program follows (check the Requirements section for specifics on program modularization):

1. **Introduce** the program and programmer, and describe the program functionality.
2. **Declare** global constants `DAYS_MEASURED`, `TEMPS_PER_DAY`, `MIN_TEMP` and `MAX_TEMP`.
3. **Generate** `ARRAYSIZE` random integers in the range from `MIN_TEMP` to `MAX_TEMP` (inclusive), storing

them in consecutive elements of array `tempArray`. (e.g. for `MIN_TEMP` = 20 and `MAX_TEMP` = 80, generate values from the set [20, 21, ... 80])
   - `DAYS_MEASURED` should be initially set to 14
   - `TEMPS_PER_DAY` should be initially set to 11
   - `MIN_TEMP` should be initially set to 20
   - `MAX_TEMP` should be initially set to 80
   - *Hint:* Call `Randomize` once at the beginning of `main` to generate a random seed. Later, use `RandomRange` to generate each random number.
4. **Find** the highest temperature of each day. Store these values into an array of length `DAYS_MEASURED`.
5. **Find** the lowest temperature of each day. Store these values into a different array of length `DAYS_MEASURED`.
6. **Calculate** the (truncated) average of the array of high temperatures (storing this into a variable) and the (truncated) average of the array of low temperatures (storing this into another variable).
7. **Display** the list of temperature values, with one day's values printed on each line and one space between each value, and a descriptive title.
8. **Display** the list of daily high temperatures with one space between each value, and a descriptive title.
9. **Display** the list of daily low temperatures with one space between each value, and a descriptive title.
10. **Display** the average high temperature with a descriptive title.
11. **Display** the average low temperature with a descriptive title.

Program Requirements

1. The program **must** be constructed using procedures. *At least* the following procedures/ parameters are required:
   NOTE: Regarding the syntax used below:
   `procName` {parameters: *varA* (value, input), *varB* (reference, output)} indicates that procedure `procName` must be passed *varA* as a value and *varB* as a reference, and that *varA* is an input parameter and *varB* is an output parameter. *You may use more parameters than those specified but try to only use them if you **need** them.*
   a. `main`
   b. `printGreeting` {parameters: *intro1* (reference, input), *intro2* (reference, input), ...)
   You should also use this procedure for a farewell message, if you include one.
   c. `generateTemperatures` {parameters: *tempArray* (reference, output)}
   MIN_TEMP, MAX_TEMP, DAYS_MEASURED, TEMPS_PER_DAY will be used as globals within this procedure.
   d. `findDailyHighs` {parameters: *tempArray* (reference, input), *dailyHighs* (reference, output)}
   DAYS_MEASURED and TEMPS_PER_DAY will be used as globals within this procedure.
   e. `findDailyLows` {parameters: *tempArray* (reference, input), *dailyLows* (reference, output)}

DAYS_MEASURED and TEMPS_PER_DAY will be used as globals within this procedure.

f. `calcAverageLowHighTemps` {parameters: *dailyHighs* (reference, input), *dailyLows* (reference, input), *averageHigh* (reference, output), *averageLow* (reference, output) }
DAYS_MEASURED will be used as a global within this procedure.

g. `displayTempArray` {parameters: *someTitle* (reference, input), *someArray* (reference, input), *someRows* (reference, input), *someColumns* (reference, input) }
You will use this to print the big temperature array and also the low- and high- temperature arrays. Be careful to construct it so it can print "someRows" rows and each row will have "someColumn" elements, even if the number of rows or columns is 1.

h. `displayTempwithString` {parameters: *someTitle* (reference, input), *someValue* (value, input)}
You will use this to print both the average high and the average low temperature. It should print a string and then a decimal value.

2. Procedures (except `main`) **must not** reference data segment variables by name. There is a **significant** penalty attached to violations of this rule. `tempArray`, `dailyHighs`, `dailyLows`, titles for the arrays, etc... should be declared in the .data preceding `main`, but **must** be passed to procedures on the stack.
   - Constants `DAYS_MEASURED`, `TEMPS_PER_DAY`, `MIN_TEMP` and `MAX_TEMP` may be used by name in procedures as needed, since they are global constants.
   - Parameters **must** be passed on the system stack, by value or by reference, as noted above (see **Module 7, Exploration 1 - Passing Parameters on the Stack (https://canvas.oregonstate.edu/courses/1976334/pages/exploration-1-passing-parameters-on-the-stack)** for method).
   - Strings/arrays **must** be passed by reference.
3. Since you will not be using globals (except the constants) the program **must** use one (or more) of the addressing modes from the explorations (e.g. *Register Indirect* or *Indirect Operands* addressing for reading/writing array elements, and *Base+Offset* addressing for accessing parameters on the runtime stack.)
   See **Module 7, Exploration 2 - Arrays in Assembly and Writing to Memory (https://canvas.oregonstate.edu/courses/1976334/pages/exploration-2-arrays-in-assembly-and-writing-to-memory)** for details.
4. The programmer's name and program title, and a description of program functionality (in student's own words) to the user must appear in the output.
5. `DAYS_MEASURED`, `TEMPS_PER_DAY`, `MIN_TEMP` and `MAX_TEMP` **must** be declared as constants.
   NOTE: We **will** be changing these constant values to ensure they are implemented correctly. Expect ranges as follows
   `MIN_TEMP`: 10 to 30
   `MAX_TEMP`: 40 to 99
   `DAYS_MEASURED`: 1 to 28
   `TEMPS_PER_DAY`: 1 to 23

6. There **must** be only one procedure to display arrays. This procedure **must** be called three times:
   i. to display the array of all temperatures;
   ii. to display the array of daily high temperature measurements;
   iii. to display the array of daily low temperature measurements.
7. All lists **must** be identified when they are displayed (use the *someTitle* parameter for the `displayTempArray` procedure).
8. Procedures **may** use local variables when appropriate.
9. The program **must** be fully documented and laid out according to the **CS271 Style Guide (https://canvas.oregonstate.edu/courses/1976334/files/107292405/download?wrap=1)** . This includes a complete header block for identification, description, etc., a comment outline to explain each block of code, and proper procedure headers/documentation.

Notes

1. You are now allowed to use the `USES` directive, but only for preserving/restoring registers. You may not use its extended syntax to create local labels for stack-passed parameters.
2. The Irvine library provides procedures for generating random numbers. Call `Randomize` once at the beginning of the program (to set up so you don't get the same sequence every time), and call `RandomRange` to get a pseudo-random number. See the documentation in the **CS271 Irvine Procedure Reference (https://canvas.oregonstate.edu/courses/1976334/files/107292415/download?wrap=1)** for more information on using these procedures.
3. If you are getting strange array/string printing output, some versions of the windows terminal can be causing this issue. To test if this is what's happening and maybe remedy the issue:
   Set a breakpoint on your program exit statement (usually Invoke Exitprocess,0) then "Start Debugging". Compare your program behavior running this way, to your program behavior running it with "Start without Debugging". If the two outputs are different, it's most likely the bug. When we grade, we will be checking both ways of running your program, so if your program only behaves correctly with the workaround you'll be just fine! We have been able to link it to the Windows 11 terminal version 1.15.2875.0. Some students have also reported the bug with version 1.16.10262.0.
   Here's a fix that some students last term had success with:
   1. Go to Settings (The Windows app) -> Privacy & Security -> For Developers -> Terminal
   2. From there, change Terminal settings from Let Windows Decide to Windows Console Host.
4. Check the **Course Syllabus (https://canvas.oregonstate.edu/courses/1976334/files/107957235/download?wrap=1)** for late submission guidelines.
5. Find the assembly language instruction syntax and help in the **CS271 Instructions Reference (https://canvas.oregonstate.edu/courses/1976334/files/107642447/download?wrap=1)** .
6. To create, assemble, run, and modify your program, follow the instructions on the course **Syllabus Page (https://canvas.oregonstate.edu/courses/1976334/assignments/syllabus)** 's "Tools" tab.

Resources

Additional resources for this assignment

- **Project Shell with Template.asm (https://canvas.oregonstate.edu/courses/1976334/files/107292165/download?wrap=1) (https://canvas.oregonstate.edu/courses/1976334/files/107292165/download?download_frd=1)**
- **CS271 Style Guide (https://canvas.oregonstate.edu/courses/1976334/files/107292405/download?wrap=1)**
- **CS271 Instructions Reference (https://canvas.oregonstate.edu/courses/1976334/files/107642447/download?wrap=1)**
- **CS271 Irvine Procedure Reference (https://canvas.oregonstate.edu/courses/1976334/files/107292415/download?wrap=1)**

What to turn in

Turn in a single .asm file (the actual Assembly Language Program file, not the Visual Studio solution file). File must be named "Proj5_ONID.asm" where ONID is your ONID username. Failure to name files according to this convention may result in reduced scores (or ungraded work).

Example Execution

```
Welcome to Chaotic Temperature Statistics, by Lunacy Lovelace
This program generates a series of temperature readings, X per day for Y days
(depending on CONSTANTs), and performs some basic statistics on them: daily h
igh and low
and average high and low temps. It then prints these results, with descriptiv
e titles.

The temperature readings are as follows (one row is one day):
75  69  57  62  77  75  45  21  64  79  44
25  24  69  32  47  51  41  21  49  77  43
78  28  47  79  41  60  35  49  51  49  70
24  60  55  36  40  25  75  77  41  67  67
33  40  30  27  58  35  31  22  22  45  65
66  54  43  34  29  69  42  59  48  30  64
56  23  58  72  70  43  78  36  50  28  61
30  48  33  73  71  37  39  32  69  59  76
79  37  38  25  53  30  45  23  52  47  31
57  52  57  71  20  79  30  73  33  50  40
62  74  67  52  31  50  64  44  72  40  23
42  63  78  33  46  24  28  50  28  55  56
28  28  24  37  67  39  30  52  43  59  61
44  25  64  29  56  80  51  58  37  20  48
```

```
The highest temperature of each day was:
79  77  79  77  65  69  78  76  79  79  74  78  67  80

The lowest temperature of each day was:
21  21  28  24  22  29  23  30  23  20  23  24  24  20

The (truncated) average high temperature was: 75

The (truncated) average low temperature was: 23

Thanks for using Chaotic Temperature Statistics. Goodbye!
```

Extra Credit (Original Project Definition must be Fulfilled)

To receive points for any extra credit options, you **must** add one print statement to your program output **per extra credit** which describes the extra credit you chose to work on. You ***will not receive extra credit points*** unless you do this. The statement must be formatted as follows...

```
--Program Intro--
**EC: DESCRIPTION
--Program prompts, etc—
```

For example, for extra credit option #2, program execution would look like this:

```
Welcome to Chaotic Temperature Statistics, by Lunacy Lovelace
**EC: Temperatures are generated such that each day starts cold, gradually gr
ows warmer until around noon, and then becomes colder until the end of the da
y.

This program generates a series of temperature readings, X per day for Y days
(depending on CONSTANTs), and performs some basic statistics on them: daily h
igh and low
and average high and low temps. It then prints these results, with descriptiv
e titles.
...
...
```

Extra Credit Options

1. Display the original temperature array so that each column corresponds to one day, rather than each row corresponding to one day. (1pt)
2. Generate temperature values on a diurnal cycle so that they actually make sense.
   Each day should start with a fairly low temperature, then increase until the middle of the day, then decrease until the end of each day. The next day should pick up where the preceding day left off. `MIN_TEMP` and `MAX_TEMP` bounds should still be respected, and there must be random variation in the change of temperature from one reading to the next. (4pts)

Grading criteria

Please view the rubric attached to this assignment to understand how your assignment will be graded. If you have any questions please ask on the course discussion board.

Project 5 Rubric

Criteria	Ratings			Pts
Files Correctly Submitted Submitted file is correct assignment and is an individual .asm file.	**1 pts** **Full Marks**		**0 pts** **No Marks**	1 pts
Program Assembles, Links, & Executes Submitted program assembles, links/ loads, executes without need for clarifying work for TA and/or messages to the student. The program must be a legitimate attempt at the assignment. Non-attempts which compile/link/run earn no points.	**3 pts** **Full Marks**		**0 pts** **No Marks**	3 pts
Documentation - Identification Block - Header Name, Date, Program number, etc as per Style Guide are included in Identification Block	**1 pts** **Full Marks**		**0 pts** **No Marks**	1 pts
Documentation - Identification Block - Program Description Description of functionality and purpose of program is included in identification block, in student's own words.	**1 pts** **Full Marks**		**0 pts** **No Marks**	1 pts
Documentation - Procedure Headers Procedure headers as per Style Guide: Describe functionality and implementation of program flow; list pre- and post-conditions and registers changed; ...	**3 pts** **Full Marks**	**2 pts** **Headers without Conditions** Descriptive headers but lacking pre- and post-conditions and 'registers changed'	**0 pts** **No Marks** Headers not descriptive, no pre-post conditions and no 'registers changed'	3 pts

Criteria	Ratings				Pts
Documentation - Block and In-line Comments Block and In-line comments contribute to understanding of program flow where necessary, but are not line-by-line descriptions of moving memory to registers. See CS271 Style Guide.	**1 pts** **Full Marks**		**0 pts** **No Marks**		1 pts
Completeness - Displays Title & Programmer Name, and Program Description Program prints out the programmer's name and Program Title. Program description describes functionality of the program to the user in the student's own words.	**1 pts** **Full Marks**		**0 pts** **No Marks**		1 pts
Completeness - Displays Full Temperature Array	**2 pts** **Full Marks**		**0 pts** **No Marks**		2 pts
Completeness - Displays Lists of Daily High and Low Temperatures	**2 pts** **Full Marks**	**1 pts** **Missing one** Missing either the daily high temp array or the daily low temp array.		**0 pts** **No Marks**	2 pts
Completeness - displayTempArray prints arrays with a descriptive title Must include 'Temperatures', 'high temperature', 'low temperature' for the three arrays, respectively. No	**3 pts** **Full Marks**	**2 pts** **One Title Missing**	**1 pts** **Two Titles Missing**	**0 pts** **No Marks**	3 pts

Criteria	Ratings				Pts
points if the title is not printed by displayTempArray					
Completeness - Displays Average High and Low Temperatures	**2 pts** **Full Marks**	**1 pts** **Missing One** Either average high or average low temp is missing.		**0 pts** **No Marks**	2 pts
Correctness - Array declarations are sized based on the constant values TEMPS_PER_DAY and DAYS_MEASURED Changing the CONSTANT value assignments correctly alters program functionality.	**2 pts** **Full Marks**	**1 pts** **Most but not All** Most arrays are declared correctly but not all of them are.		**0 pts** **No Marks**	2 pts
Correctness - Generated random numbers are in the correct range. Range is specified by MIN_TEMP and MAX_TEMP	**4 pts** **Full Marks**	**3 pts** **Edge Case Issues** Edge cases are missing, or the numbers can exceed the range by one value.	**2 pts** **Ignores MIN_TEMP** Generated values are in the [0,...,MAX_TEMP] range, rather than [MIN_TEMP,...MAX_TEMP] range.	**0 pts** **No Marks**	4 pts
Correctness - Generates the correct number of temperature values. There should be (DAYS_MEASURED * TEMPS_PER_DAY) total temperature values generated.	**3 pts** **Full Marks**			**0 pts** **No Marks**	3 pts
Correctness - Temperature values are correctly stored in tempArray array.	**3 pts** **Full Marks**			**0 pts** **No Marks**	3 pts

Criteria	Ratings				Pts
Correctness - displayTempArray displays correct ordering. Temperature values are displayed in the correct order, and the display is correct for input parameters (number of rows and number of columns)	**4 pts** **Full Marks**	**3 pts** **High/Low Incorrect** The original array is displayed correctly but the daily high and/or daily low temperature arrays are not.	**2 pts** **Main Array Incorrect** The daily high/ low arrays display correctly but the original array does not.	**0 pts** **No Marks**	4 pts
Correctness - Daily High/Low Temperatures The arrays holding the daily high and low temperatures are correct.	**3 pts** **Full Marks**	**2 pts** **Mostly Correct** Some high or low temperature values are incorrect		**0 pts** **No Marks**	3 pts
Correctness - Average High/Low Temperatures The average high and low temperatures are correctly calculated from the daily high and low temperature arrays.	**2 pts** **Both Correct**	**1 pts** **One Incorrect**	**0 pts** **Both Incorrect**		2 pts
Coding Style - Modular Design Uses at least procedures: main, printGreeting, generateTemperatures, findDailyHighs, findDailyLows, calcAverageLowHighTemps, displayTempArray, displayTempWithString. Student may combine findDailyHighs and findDailyLows	**5 pts** **Full Marks**		**0 pts** **No Marks**		5 pts
Coding Style - Uses appropriately named identifiers Identifiers named so that a person reading the code can intuit the purpose of a variable, constant, or label just by reading its name.	**2 pts** **Full Marks**		**0 pts** **No Marks**		2 pts

Criteria	Ratings		Pts
(CS271 Style Guide)			
Coding Style - Readability Program uses readable white-space, indentation, and spacing as per the CS271 Style Guide. Logical blocks are separated by white space.	**1 pts** **Full Marks**	**0 pts** **No Marks**	1 pts
Output Style - Readability Program output is easy to read	**1 pts** **Full Marks**	**0 pts** **No Marks**	1 pts
Requirements - No global variable references outside of main Deduct 5 points per procedure where any data segment identifiers (variables) are referenced by name (other than MAIN). Maximum deduction can only drop the final program score to 0.	**0 pts** **Full Marks**	**0 pts** **No Marks**	0 pts
Requirements - displayTempArray used for all array printouts Three different CALLs to the same displayTempArray procedure (one for the original full temperature measurement list, and one each for the daily high temperatures and daily low temperatures. Deduct 5 points if there is more than one array-display procedure.	**0 pts** **Full Marks**	**0 pts** **No Marks**	0 pts
Requirements - Correct Array Addressing Modes Deduct 5 points if Register Indirect (register as pointer), Indirect Operands, or Base+Offset addressing not used for array/stack parameter access.	**0 pts** **Full Marks**	**0 pts** **No Marks**	0 pts
Extra Credit Displays the temperatures ordered by column (day) instead of by row (day) (1) |	**0 pts** **Full Marks**	**0 pts** **No Marks**	0 pts

Criteria	Ratings		Pts
Temperature values are generated on a diurnal cycle. (4)			
Late Penalty Remove points here for late assignments. (Enter negative point value)	**0 pts** **Full Marks**	**0 pts** **No Marks**	0 pts
			Total Points: 50